TECHNICAL DOCUMENT · DEVELOPER REFERENCE

NUVI Social Media Advertisement Distribution Manager

The developer's reference for building Version 1: data fields, validation rules, file formats, engine logic, screen behaviour, messages and tests.

BASED ON

BRD v1.0 · SAD v1.0

Business Requirements · System Design & Architecture

PREPARED BY

Amritha Hitige

NUVI Engineering (Pvt) Ltd

STACK

Chrome Extension MV3 · Vanilla JS

Side Panel · Service Worker · chrome.storage.local

DATE / VERSION

13 November 2026 · v1.0

Draft for review

CONTENTS

01 Introduction & Conventions 2

02 Constants & Default Settings 3

03 Data Dictionary 4

04 Validation & URL Rules 5

05 Storage Module API 6

06 CSV & Backup Formats 7

07 Distribution Engine 8

08 Message API 9

09 Screen Specifications 10

10 Messages Catalogue 11

11 Testing 12

12 Coding Standards & Release 12

01 · INTRODUCTION & CONVENTIONS

Purpose of this document

The BRD defines **what** to build and the SAD defines the **structure**. This document gives developers the exact **details**: field rules, formats, algorithms and test cases. It is short on purpose, because this is a light application.

Reference documents

Document	Use
BRD v1.0 (13 Nov 2026)	Requirements FR-001–043, rules BR-001–019, NFR-01–10, AC-001–022
System Design & Architecture v1.0	Components, storage keys, state machine, flows, permissions

Technology summary

Item	Decision
Platform	Google Chrome desktop, **version 116+**, Manifest V3
Language	JavaScript ES2022, ES modules (`import` / `export`)
UI	Plain HTML + CSS in the Chrome Side Panel; no framework
External libraries	**None** in Version 1 (CSV parser and helpers are written in-house)
Storage	`chrome.storage.local` + `unlimitedStorage`
Build step	None; the folder is loaded unpacked. Optional: a zip for distribution
Tests	Plain JS test runner page (`tests/index.html`) for `shared/` and engine logic

Conventions

Topic	Rule
Dates stored	ISO 8601 in UTC, e.g. `2026-11-13T03:50:00.000Z` (`new Date().toISOString()`)
Dates displayed	`DD/MM/YYYY HH:mm` in the computer's local time (Asia/Colombo)
Enum values	`UPPER_SNAKE_CASE` in storage; friendly labels only in the UI
File names	`kebab-case.js` ; one module per file
Functions / variables	`camelCase` ; constants `UPPER_SNAKE_CASE`
Text input	Always `trim()` ; collapse repeated spaces in names
Language	English UI; Sinhala text supported everywhere (UTF-8)
Phases	P1 MVP · P2 Campaign management (as in the BRD)

02 · CONSTANTS & DEFAULT SETTINGS

Fixed values used everywhere

These are defined once in `shared/constants.js` and imported where needed.

```
export const BUSINESSES = {
  B001: { id: 'B001', name: 'NUVI Engineering (Pvt) Ltd', short: 'NUVI Engineering', color: '#d97706' },
  B002: { id: 'B002', name: 'NUVI IT (Pvt) Ltd',          short: 'NUVI IT',          color: '#6d28d9' }
};
export const GROUP_STATUS    = ['ACTIVE', 'INACTIVE'];
export const AD_STATUS       = ['ACTIVE', 'COMPLETED', 'ARCHIVED'];
export const CAMPAIGN_STATUS = ['IN_PROGRESS', 'PAUSED', 'STOPPED', 'COMPLETED'];
export const ITEM_STATUS     = ['PENDING', 'IN_PROGRESS', 'PUBLISHED', 'SKIPPED',
                                'APPROVAL_REQUIRED', 'FAILED'];
export const FINAL_STATUS    = ['PUBLISHED', 'SKIPPED', 'APPROVAL_REQUIRED', 'FAILED'];
export const SCHEMA_VERSION  = 1;
export const LIMITS = { groupName: 150, adName: 120, notes: 500, description: 500,
                        reason: 200, location: 80, csvRows: 1000, groupsPerBusiness: 1000 };
```

Status labels and colours (UI)

Value	Label	Badge colour
`PENDING`	Pending	Grey #94a3b8
`IN_PROGRESS`	In Progress	Blue #2563eb
`PUBLISHED`	Published	Green #15803d
`SKIPPED`	Skipped	Slate #64748b
`APPROVAL_REQUIRED`	Approval Required	Amber #d97706
`FAILED`	Failed	Red #b91c1c

Default settings (first install)

```
export const DEFAULT_SETTINGS = {
  categories: ['Property', 'Location-Based', 'Business', 'Community', 'Industry', 'Other'],
  adTypes: {
    B001: ['House', 'Land', 'Construction', 'Property Project', 'Other'],
    B002: ['Software', 'Website', 'Inventory System', 'Business Software', 'Other']
  },
  locations: [],               // grows automatically from entered group locations
  postingGapSeconds: 0,        // P2 - 0 = off (FR-039)
  recentPostWarningDays: 0,    // P2 - 0 = off (FR-039)
  backupReminderDays: 7        // FR-041
};
```

First install: on `chrome.runtime.onInstalled` with `reason === 'install'`, write `meta` (schemaVersion 1, counters 0), `settings` = defaults, empty `groups` / `ads` / `campaigns`, and `session = null`. On `'update'`, run migrations only (Section 05).

03 · DATA DICTIONARY

Every stored field

Group (key: `groups`, object by id)

Field	Type	Req.	Rule
id	string	auto	`GRP-0001` … never reused
businessId	string	✓	`B001` | `B002`; set from the selected business, cannot be edited
name	string	✓	1–150 chars, trimmed
url	string	✓	Canonical form `https://www.facebook.com/groups/{id}/` (Section 04)
urlKey	string	auto	`facebook.com/groups/{id}` (lower case); unique per business
category	string	–	One of `settings.categories`; default `Other`
location	string	–	≤ 80 chars; new values added to `settings.locations`
status	enum	✓	`ACTIVE` (default) | `INACTIVE`
notes	string	–	≤ 500 chars
createdAt / updatedAt	ISO date	auto	Set on create / every save

Advertisement (key: `ads`)

Field	Type	Req.	Rule
id	string	auto	`AD-0001` …
businessId	string	✓	From the selected business; cannot be edited
name	string	✓	1–120 chars; unique per business (case-insensitive)
postUrl	string	✓	A valid Facebook post URL (Section 04); stored as entered, without tracking parameters
type	string	–	One of `settings.adTypes[businessId]`
description	string	–	≤ 500 chars
status	enum	✓	`ACTIVE` (default) | `COMPLETED` | `ARCHIVED`; only ACTIVE ads can be distributed
createdAt / updatedAt	ISO date	auto	createdAt can be edited (date only)

Campaign & Item (key: `campaigns`)

Field	Type	Rule
id	string	`CAM-001` … (3+ digits)
businessId, adId, adName, adUrl	string	From the advertisement; name and URL are a **snapshot** at Start (BR-017)
status	enum	`IN_PROGRESS | PAUSED | STOPPED | COMPLETED`
currentIndex	number	Index of the item now open (−1 when finished)
startedAt / completedAt	ISO date	completedAt is null until COMPLETED
items[]	array	One per selected group, in selection order; no duplicate groupId (BR-016)
· groupId / Name / Url	string	Snapshot of the group at Start
· status, reason	enum / string	Item status; reason ≤ 200 chars (Approval / Failed only)
· log[]	array	`{s: status, at: ISO, r?: reason}`, one entry per change

session = `{campaignId, tabId, originalTabId, openedAt}` or `null` · **meta** = `{schemaVersion, counters:{group, ad, campaign}, lastBackupAt, lastBusinessId}`

URL validation and normalization

Group URL (FR-004, FR-010)

```
// accepted: http/https, optional www. / m. / web. / mbasic., any query or hash
const GROUP_RE = new RegExp(
  '^https?://(?:(?:www|m|web|mbasic)\\.)?facebook\\.com/groups/' +
  '([A-Za-z0-9._-]+)/?(?:[?#].*)?$', 'i');

export function parseGroupUrl(input) {
  const raw = (input || '').trim();
  const m = raw.match(GROUP_RE);
  if (!m) return { ok: false, error: 'E-URL-01' };
  const gid = m[1].toLowerCase();
  return { ok: true,
           url: `https://www.facebook.com/groups/${gid}/`,   // stored + used for navigation
           urlKey: `facebook.com/groups/${gid}` };            // duplicate check key
}
```

Input	urlKey	Valid
https://www.facebook.com/groups/233212924715137/	facebook.com/groups/233212924715137	✓
http://m.facebook.com/groups/233212924715137?ref=share	facebook.com/groups/233212924715137	✓ (duplicate of above)
https://web.facebook.com/groups/HomagamaApi	facebook.com/groups/homagamaapi	✓
https://www.facebook.com/groups/2744442632268081/posts/123/	—	× (post link, not a group)
https://www.facebook.com/NUVIEngineering	—	× (page, not a group)
(empty)	—	× E-REQ-01

Advertisement post URL (FR-015)

```
const POST_RE = /^https?:\/\/(?:(?:www|m|web|mbasic)\.)?(?:facebook\.com|fb\.com)\/.+/i;
// Valid examples: facebook.com/NUVIEngineering/posts/pfbid0..., facebook.com/share/p/AbC123/,
//                 facebook.com/permalink.php?story_fbid=...&id=..., facebook.com/photo/?fbid=...
// Before saving: remove tracking params (fbclid, __cft__, __tn__, mibextid); keep story_fbid, id, fbid, v.
```

Other field rules

Check	Rule	Code
Required field empty	After trim, length = 0	E-REQ-01
Too long	Over LIMITS value (Section 02)	E-LEN-01
Duplicate group	Same `urlKey` already in **this business** (other business allowed)	E-DUP-01
Duplicate ad name	Same name (case-insensitive) in this business	E-DUP-02
Group limit	More than 1,000 groups in one business	E-LIM-01
Unknown category / type	Not in the settings list → saved as `Other` (CSV) or blocked (form)	W-CAT-01

05 · STORAGE MODULE API

shared/storage.js

This is the only module that calls `chrome.storage.local`. Every function is `async`.

Function	Behaviour
`listGroups(businessId)`	Array of groups of one business, sorted by name
`saveGroup(group)`	Validate → set id / dates → write; returns the saved group or throws an error code
`deleteGroup(id)`	Remove the group (campaign history keeps its snapshot)
`importGroups(businessId, rows)`	Write all valid rows in **one** `set()`; returns `{added}`
`listAds(businessId)` / `saveAd(ad)` / `deleteAd(id)`	Same pattern. `deleteAd` refuses if campaigns exist → suggests Archive (FR-013)
`listCampaigns(businessId)`	Newest first
`getCampaign(id)` / `saveCampaign(c)`	Used by the engine (and by history for FR-038)
`getSession()` / `setSession(s)`	Active session pointer or `null`
`getSettings()` / `saveSettings(s)`	Merged with `DEFAULT_SETTINGS` for missing keys
`nextId(kind)`	Increments `meta.counters[kind]` → `GRP-0001` / `AD-0001` / `CAM-001`
`publishedIndex(adId, excludeCampaignId?)`	Set of groupIds with PUBLISHED or APPROVAL_REQUIRED for this ad, across all campaigns (FR-026)
`lastPostedAt(groupId)`	Latest PUBLISHED time for any ad (P2, FR-039)
`exportAll()` / `importAll(data)`	Backup / restore (Section 06)
`migrate()`	Runs `MIGRATIONS[v]` from the stored to the current `SCHEMA_VERSION`

Write ownership (prevents conflicting updates)

Key	Writer
`groups`, `ads`, `settings`	Side panel only
`campaigns`, `session`	Service worker only. Exception: History status correction (FR-038), allowed only when no session is active
`meta`	Both, via `nextId()` (read-modify-write in one call)

Example

```
export async function saveGroup(input) {
  const { groups = {} } = await chrome.storage.local.get('groups');
  const p = parseGroupUrl(input.url);
  if (!p.ok) throw new AppError(p.error);
  const dup = Object.values(groups).find(g =>
      g.businessId === input.businessId && g.urlKey === p.urlKey && g.id !== input.id);
  if (dup) throw new AppError('E-DUP-01', { name: dup.name });
  const now = new Date().toISOString();
  const g = { ...groups[input.id], ...clean(input), url: p.url, urlKey: p.urlKey,
              id: input.id || await nextId('group'), updatedAt: now };
  g.createdAt ??= now;  groups[g.id] = g;
  await chrome.storage.local.set({ groups });  return g;
}
```

File formats

Group CSV, import and export (FR-010, FR-031)

Rule	Value
Encoding	UTF-8 (a leading BOM is accepted and removed). Export writes UTF-8 **with BOM**, so Excel shows Sinhala correctly
Delimiter / quotes	Comma; fields may be wrapped in double quotes; `""` inside quotes = one quote
Header row	Required, case-insensitive, any column order
Columns	`group_name` * , `group_url` * , `category` , `location` , `status` , `notes`
Business	Not in the file: the rows go to the **currently selected** business
status	Active / Inactive (any case); empty = Active
Limits	Max 1,000 data rows and 1 MB per file; empty lines ignored

```
group_name,group_url,category,location,status,notes
Kiriwaththuduwa Api,https://www.facebook.com/groups/233212924715137/,Location-Based,Kiriwaththuduwa,Active,
Homagama Api,https://www.facebook.com/groups/2744442632268081/,Location-Based,Homagama,Active,"Post link only"
Colombo Api,https://www.facebook.com/groups/853336822316464/,Location-Based,Colombo,Active,
```

Row result	When
VALID	All checks pass. It will be imported
INVALID	Missing name or URL, bad group URL, or too long. The reason is shown
DUPLICATE	`urlKey` already exists in this business, or appears earlier in the same file

Backup file (FR-041, FR-042)

```
// file name: nuvi-distribution-backup-2026-11-13.json
{
  "app": "nuvi-distribution-manager",
  "schemaVersion": 1,
  "exportedAt": "2026-11-13T06:30:00.000Z",
  "appVersion": "1.0.0",
  "data": { "meta": {...}, "settings": {...}, "groups": {...},
            "ads": {...}, "campaigns": {...} }        // session is NOT exported
}
```

#	Restore steps
1	Refuse if a session is active (E-BAK-03)
2	Parse JSON; check `app` name and that all 5 data keys exist (E-BAK-01)
3	If `schemaVersion` > current → refuse (E-BAK-02); if lower → run migrations on the data in memory
4	Confirm "Replace all current data?" → one `set()` with all keys + `session = null`

07 · DISTRIBUTION ENGINE

background/distribution-engine.js

Each message handler **loads state from storage first** and **saves before navigating**. No state is kept in memory between messages (the MV3 service worker may sleep).

```
export async function start({ adId, groupIds }, sender) {
  if (await getSession()) throw new AppError('E-CAM-01');           // BR-018
  const ad = await getAd(adId);
  if (!ad || ad.status !== 'ACTIVE') throw new AppError('E-CAM-02');
  const groups = await getGroups([...new Set(groupIds)]);           // BR-016 de-duplicate
  if (!groups.length) throw new AppError('E-CAM-03');
  if (groups.some(g => g.businessId !== ad.businessId)) throw new AppError('E-CAM-04'); // BR-013
  const now = iso();
  const c = { id: await nextId('campaign'), businessId: ad.businessId, adId,
              adName: ad.name, adUrl: ad.postUrl, status: 'IN_PROGRESS', currentIndex: 0,
              startedAt: now, completedAt: null,
              items: groups.map(g => ({ groupId: g.id, groupName: g.name, groupUrl: g.url,
                                        status: 'PENDING', reason: null, log: [] })) };
  setItem(c.items[0], 'IN_PROGRESS');
  await saveCampaign(c);
  await setSession({ campaignId: c.id, tabId: await activeTabId(), openedAt: now });
  await openCurrent(c);                                    // tab-manager: tabs.update / create
  return state();
}

export async function result({ status, reason }) {          // PUBLISHED | SKIPPED | APPROVAL_REQUIRED | FAILED
  const { c } = await loadActive('IN_PROGRESS');
  setItem(c.items[c.currentIndex], status, reason);
  const next = nextPending(c);
  if (next === -1) return complete(c);                     // FR-037
  c.currentIndex = next;
  setItem(c.items[next], 'IN_PROGRESS');
  await saveCampaign(c);                                   // save BEFORE navigation
  if ((await publishedIndex(c.adId, c.id)).has(c.items[next].groupId))
    return { ...(await state()), confirmDuplicate: c.items[next].groupName }; // FR-026 (P2)
  await openCurrent(c);
  return state();
}

export async function previous() {
  const { c } = await loadActive('IN_PROGRESS');
  if (c.currentIndex === 0) throw new AppError('E-CAM-05');
  setItem(c.items[c.currentIndex], 'PENDING');
  c.currentIndex -= 1;
  setItem(c.items[c.currentIndex], 'IN_PROGRESS');          // old status kept in log
  await saveCampaign(c); await openCurrent(c); return state();
}

const nextPending = c => {                                 // forward first, then wrap around
  const i = c.items.findIndex((it, k) => k > c.currentIndex && it.status === 'PENDING');
  return i !== -1 ? i : c.items.findIndex(it => it.status === 'PENDING');
};
function setItem(it, s, r = null) {
  it.status = s; it.reason = r;
  it.log.push({ s, at: iso(), ...(r && { r }) });
}
```

Handler	Key logic
`pause`	Campaign → PAUSED; the item stays IN_PROGRESS; the tab is not moved
`resume`	PAUSED or STOPPED → IN_PROGRESS; the current item (or the next PENDING item) is set to IN_PROGRESS; restore the session (if STOPPED); `openCurrent()`
`stop`	The current item goes back to PENDING; campaign → STOPPED; `setSession(null)`
`complete`	`currentIndex = -1`, COMPLETED, `completedAt`, `setSession(null)`; return summary counts
`openCurrent`	`tabs.update(tabId,{url, active:true})`; on error → `tabs.create({url})` and save the new `tabId`

Side panel ↔ service worker contract

```
// side panel
const res = await chrome.runtime.sendMessage({ type: 'RESULT', status: 'PUBLISHED' });
if (!res.ok) showError(res.error.code, res.error.data);

// service worker
chrome.runtime.onMessage.addListener((msg, sender, reply) => {
  HANDLERS[msg.type](msg, sender)
    .then(state => reply({ ok: true, state }))
    .catch(e => reply({ ok: false, error: { code: e.code || 'E-SYS-01', data: e.data } }));
  return true;                                  // keep the channel open for async reply
});
```

type	Request fields	Possible errors
`GET_STATE`	–	–
`START`	`adId`, `groupIds: string[]`	E-CAM-01, 02, 03, 04
`RESULT`	`status`, `reason?` (≤ 200)	E-CAM-06 (not in progress)
`PREVIOUS`	–	E-CAM-05, 06
`PAUSE`	–	E-CAM-06
`RESUME`	`campaignId?` (required for a STOPPED campaign)	E-CAM-01, 07
`STOP`	–	E-CAM-06
`CONFIRM_DUPLICATE`	`decision: 'continue' | 'skip'`	E-CAM-06
`OPEN_ORIGINAL`	–	– (re-uses `session.originalTabId` if it is still open)

State object returned to the panel

```
{
  active: true,
  campaign: { id: 'CAM-004', adName: 'Kiriwaththuduwa House for Sale', adUrl: '...',
              status: 'IN_PROGRESS', businessId: 'B001' },
  current:  { index: 2, total: 30, groupName: 'Piliyandala Property Group',
              location: 'Piliyandala', notes: 'Post link only' },   // read live from groups (FR-035)
  counts:   { PENDING: 27, IN_PROGRESS: 1, PUBLISHED: 1, SKIPPED: 1, APPROVAL_REQUIRED: 0, FAILED: 0 },
  percentPublished: 3.33,                                          // published / total x 100, 2 decimals
  confirmDuplicate: null,                                          // or group name (FR-026)
  summary: null                                                    // filled on COMPLETED (FR-037)
}
```

UI refresh: the panel also listens to `chrome.storage.onChanged` (keys `campaigns` and `session`) and calls `GET_STATE`. The screen updates even if a message reply was lost because the service worker restarted.

09 · SCREEN SPECIFICATIONS

Screen-by-screen behaviour

Side panel width is about 360–400 px: use one column, sticky action buttons at the bottom, and the business colour in the header bar.

Screen	Elements & behaviour	Phase
Header (all)	Business switch (2 buttons, coloured); the active business is saved to `meta.lastBusinessId`; a "Distribution running" chip links to Progress	P1
Dashboard	Cards: Total Groups (active / inactive), Total Advertisements (active), Active Campaigns; menu links; banners: Resume (if a session or PAUSED/STOPPED campaign exists), Backup reminder (if `lastBackupAt` is older than `backupReminderDays`)	P1
Groups list	Search box (live, name, case-insensitive, 150 ms debounce); filters: category, location, status (P2); rows: name, category, location, status badge, Edit; header: count; buttons: Add, Import CSV, Export CSV (P2)	P1
Group form	Fields as in Section 03; inline validation on blur and save; Save disabled until valid; Delete (edit mode) → confirm "Delete group ‘{name}’? Past campaign history will keep its name."	P1
CSV import	File picker (`.csv`) → preview table with Valid / Invalid / Duplicate counts and reasons → "Import {n} groups" → result toast	P1
Ads list & form	List: name, type, created date, status, "Published in n groups"; form fields per Section 03; Open link icon; Delete → Archive suggestion if history exists	P1
Distribution setup	Ad dropdown (ACTIVE only) → read-only URL; group checklist (inactive groups greyed and disabled); search; Select All (active) / Deselect All; "⚠ already published" marker; "Selected Groups: n"; START (disabled until ad + ≥1 group)	P1
Progress	Ad name; "3 / 30"; progress bar; current group name, location, notes; Copy Ad Link (toast "Link copied"); Open Original; **Published – Next Group** (green, largest); Approval Required / Failed (small reason prompt, optional); Previous, Skip, Pause / Resume, Stop (confirm). While a request is running, buttons are disabled (no double clicks)	P1
Posting gap	If `postingGapSeconds > 0`: Published-Next is disabled with a countdown "Next available in 45 s" from `session.openedAt`. If `recentPostWarningDays > 0`: a warning chip on recently posted groups	P2
Duplicate dialog	"This advertisement was already marked as published in {group} on {date}. Continue?" → Continue / Skip	P2
Summary	"Campaign completed"; counts per status; expandable lists per status with reasons; buttons: View History, Back to Dashboard	P1
History	List: ID, ad, start date, Published / Selected, status; detail: items with status, reason, time; Reuse (pre-selects the same groups, new campaign); Resume (STOPPED / PAUSED); edit item status (FR-038)	P2
Settings	Edit lists (categories, ad types per business, locations); posting gap (0–600 s); recent warning days (0–30); backup reminder days; Backup now; Restore; Reset all data (type "RESET" to confirm)	P1 / P2

10 · MESSAGES CATALOGUE

User-facing messages

All texts live in `shared/messages.js` so wording can be changed in one place.

Code	Message shown to the user
E-REQ-01	This field is required.
E-LEN-01	Too long. Maximum {max} characters.
E-URL-01	Please enter a valid Facebook group link (facebook.com/groups/...).
E-URL-02	Please enter a valid Facebook post link.
E-DUP-01	This group already exists: {name}.
E-DUP-02	An advertisement with this name already exists.
E-LIM-01	Group limit reached (1,000 per business).
E-CSV-01	The file is not a valid CSV, or the required columns (group_name, group_url) are missing.
E-CSV-02	The file is too large (max 1,000 rows / 1 MB).
E-CAM-01	{campaignId} is still in progress. Pause or stop it first.
E-CAM-02	This advertisement is not active.
E-CAM-03	Select at least one group.
E-CAM-04	Groups from another business cannot be used.
E-CAM-05	This is the first group, so there is no previous group.
E-CAM-06	No distribution is running.
E-CAM-07	This campaign cannot be resumed.
E-BAK-01	This is not a valid backup file.
E-BAK-02	This backup was made by a newer version. Please update the extension first.
E-BAK-03	Stop the running distribution before restoring a backup.
E-SYS-01	Something went wrong. Please try again. (details logged to the console)
E-SAVE-01	Could not save. Please try again.
W-CAT-01	Unknown category “{value}”, saved as Other.
C-DEL-01	Delete ‘{name}’? This cannot be undone.
C-STOP-01	Stop this distribution? Remaining groups stay Pending, and you can resume later from History.
C-RST-01	Replace ALL current data with this backup?
I-COPY-01	Advertisement link copied.
I-BAK-01	It's been {days} days since your last backup. Back up now

11 · TESTING

Unit tests (tests/index.html, run in the browser)

ID	Test
UT-01	`parseGroupUrl` : every row of the Section 04 table gives the expected result
UT-02	Post URL check: valid / invalid samples; tracking parameters removed
UT-03	CSV parser: BOM, quoted commas, `""` escapes, Sinhala names, empty lines, header in any order
UT-04	CSV validation: duplicates inside the file and against existing groups; missing columns → E-CSV-01
UT-05	Engine: START → RESULT × n → COMPLETED with correct counts and `session = null`
UT-06	Engine: PREVIOUS resets the current item to PENDING and re-opens the previous one; `nextPending` wraps around
UT-07	Engine: STOP → RESUME continues at the same group; second START refused (E-CAM-01)
UT-08	Engine: no code path sets PUBLISHED without a RESULT message (BR-009)
UT-09	Backup → restore round-trip gives identical data; a newer schema is refused
UT-10	Business separation: START with a group of the other business → E-CAM-04

The engine tests use a small fake of `chrome.storage` and `chrome.tabs` (`tests/chrome-mock.js`).

UAT

Run BRD acceptance criteria **AC-001 to AC-022** with: the real Engineering CSV (39 groups), one test CSV containing 2 invalid and 2 duplicate rows, 2 test advertisements per business, and one campaign interrupted by closing Chrome.

12 · CODING STANDARDS & RELEASE

Coding standards
ES modules, `async/await` , small pure functions in `shared/`
User text inserted with `textContent` ; never `innerHTML` with user data
No external scripts or CDNs (MV3 CSP)
Errors thrown as `AppError(code, data)` ; codes from Section 10
JSDoc on every exported function
No `console.log` in release, except errors
Git: `main` = release; feature branches; PR review

Release checklist
☐ Version bumped in `manifest.json`
☐ `SCHEMA_VERSION` + migration added if data changed
☐ Permissions unchanged (4 only)
☐ Unit tests UT-01–10 pass
☐ UAT AC-001–022 pass for this phase
☐ Users told to back up before updating
☐ Zip tagged in Git (`v1.0.0`)

Build order (from the BRD): project setup → side panel shell → business switch → groups + CSV import → advertisements → group selection → engine + tab manager → result buttons → persistence & resume → backup / restore → history (P2).